AIMS RIC — Doctoral Training School 2026

ML-ESS: A Multi-Agent AI Research Assistant

Technical Report — April 2026

Jérémie N. Landu[1]

[1]African Institute for Mathematical Sciences, Research Innovation Centre (AIMS-RIC), jeremy@aimsammi.org

Abstract

ML-ESS is a full-stack AI system that automates end-to-end research: given a natural language question, it orchestrates four specialised AI agents to search the web, synthesise evidence, draft a structured report, and evaluate the report's quality. The system exposes a REST API (FastAPI) and a modern web interface (Next.js), supporting real-time progress tracking, PDF export, and multi-provider LLM fallback. This report describes the system architecture, agent pipeline, data model, REST API, frontend application, deployment strategy, and design trade-offs.

Keywords: multi-agent systems, large language models, automated research, FastAPI, Next.js, retrieval-augmented generation.

1 Introduction & Motivation

Conducting thorough research manually is time-consuming: it involves querying multiple sources, extracting relevant claims, resolving contradictions, and composing a coherent, well-cited document. ML-ESS automates this entire workflow through a multi-agent pipeline, replacing hours of manual work with a structured, reproducible process.

Core objectives:

- Accept any research question in natural language.
- Autonomously gather evidence from the open web.
- Synthesise findings into themes, resolving contradictions.
- Generate a structured Markdown report with citations.
- Score the report quality along four dimensions.
- Deliver everything through a clean web UI with real-time feedback.

2 System Architecture

2.1 High-Level Overview

The system is divided into two main services: a Python backend responsible for all AI reasoning and data persistence, and a Next.js frontend that provides the user interface. Figure 1 illustrates the high-level architecture.

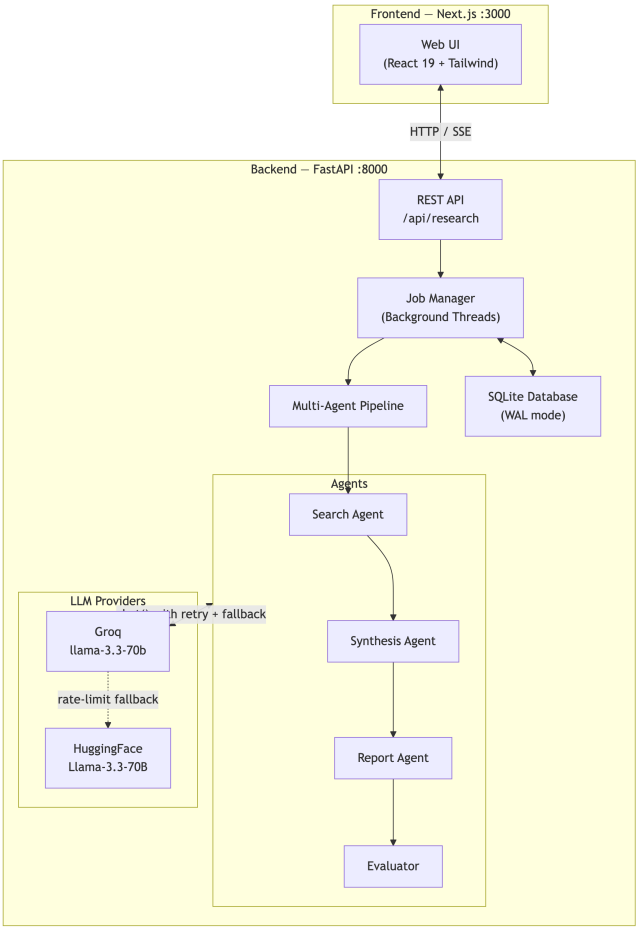


Figure 1: High-level system architecture.

2.2 Backend Directory Structure

```
api/
|-- main.py                 # CLI & server
    entry point
|-- app/
    |-- agents/
    |   |-- search.py       # Web search &
        evidence extraction
    |   |-- synthesis.py    # Theme
        identification
    |   |-- report.py       # Markdown
        generation
    |   |-- evaluator.py    # Quality scoring
    |-- api/
    |   |-- app.py          # FastAPI
        factory, CORS
```

```
|   |-- routes.py        # REST endpoints
    & SSE
|   |-- auth.py          # API key
    authentication
|-- core/
|   |-- pipeline.py      # Sequential
    orchestration
|   |-- jobs.py          # Background
    thread management
|   |-- store.py         # SQLite (WAL
    mode)
|   |-- llm.py           # LLM client w/
    retries
|-- models/
    |-- api.py           # Pydantic schemas
    |-- state.py         # Pipeline shared
        state
```

2.3 Request Lifecycle & Data Flow

Figure 2 shows the full request lifecycle from the moment a user submits a question to the delivery of the final report.

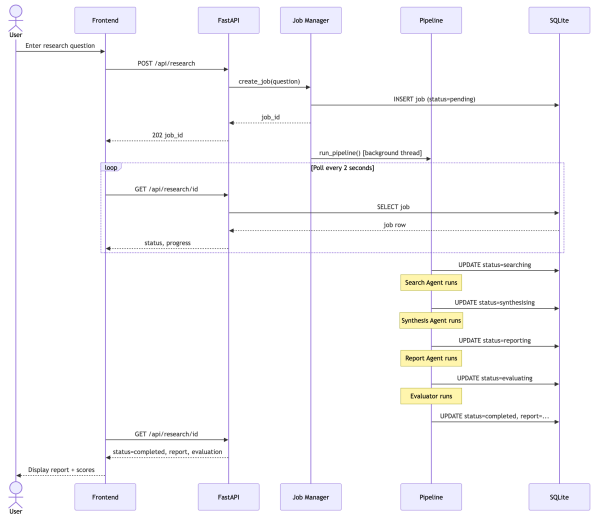


Figure 2: Request lifecycle and data flow (sequence diagram).

3 The Multi-Agent Pipeline

Each agent in the pipeline is a stateless function that reads from and writes to a shared `SharedState` object. Agents are called sequentially — each one enriches the state for the next.

3.1 SharedState Schema

Figure 3 shows the class diagram of the shared pipeline state, including the `Source`, `Evidence`, `Theme`, `Contradiction`, and `EvaluationScores` entities.

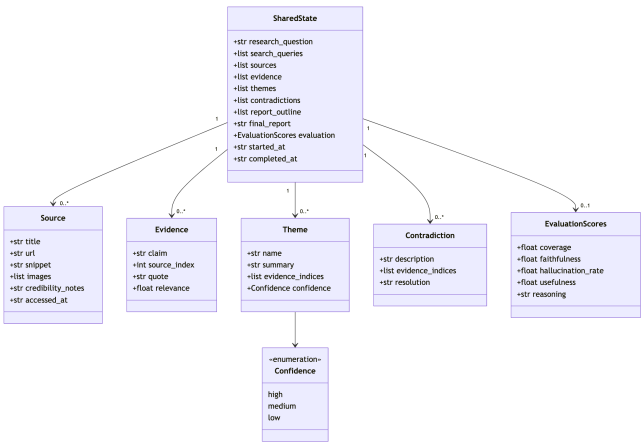
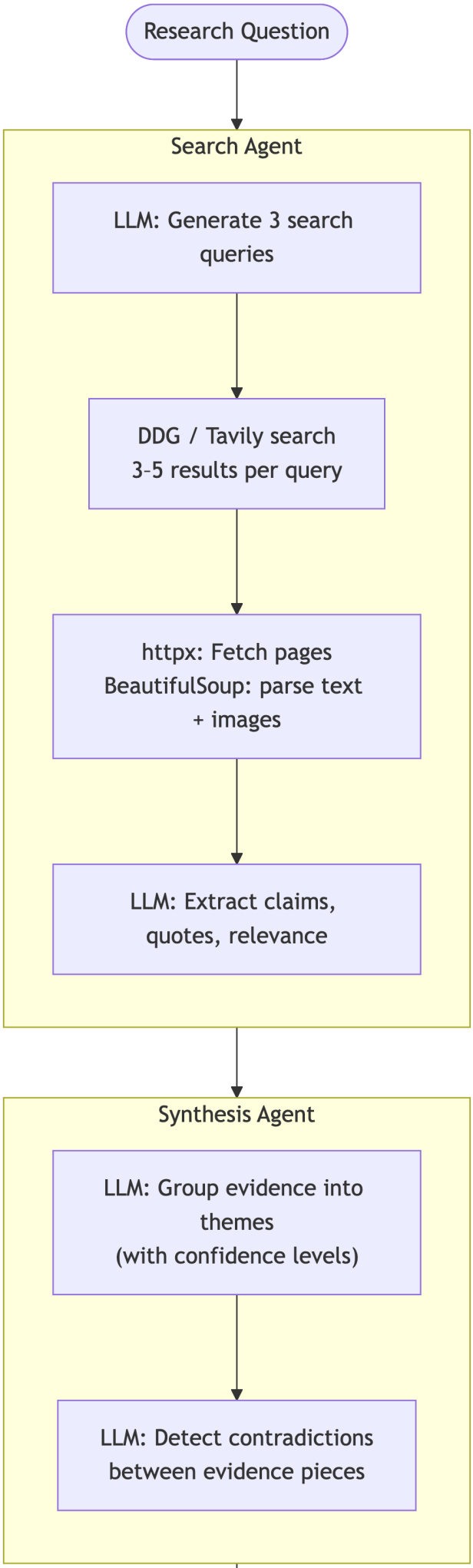


Figure 3: SharedState class diagram.

3.2 Agent Pipeline Flow

The pipeline executes four agents sequentially (Figure 4):

1. **Search Agent** — generates queries, fetches pages, and extracts structured evidence.
2. **Synthesis Agent** — groups evidence into themes and detects contradictions.
3. **Report Agent** — produces a structured Markdown report (~2 000–2 500 words) with citations.
4. **Evaluator** — scores the report on four quality dimensions.



3.3 Search Agent Detail

The Search Agent (Figure 5) generates up to three search queries via an LLM call, fans out to DuckDuckGo (or Tavily when an API key is provided), de-duplicates the result URLs, fetches page content with `httpx`, parses it with BeautifulSoup4, and finally calls the LLM again to extract structured evidence records.

Figure 5: Search Agent internal data flow.

3.4 LLM Provider Strategy

Figure 6 shows the retry and fallback strategy implemented in the LLM client. The primary provider is Groq (`llama-3.3-70b-versatile`); HuggingFace Inference (`Llama-3.3-70B-Instruct`) serves as a fallback when Groq rate-limits are exhausted.

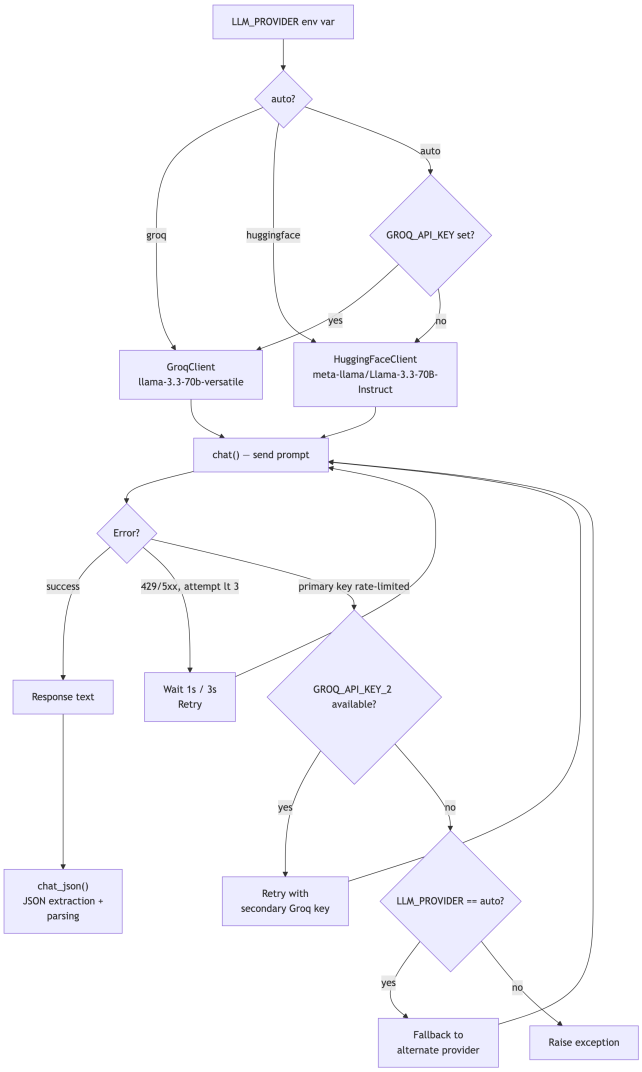


Figure 6: LLM provider retry and fallback strategy.

3.5 Evaluation Scores

The Evaluator Agent scores the generated report along four dimensions (Table 1). Scores for coverage, faithfulness, and usefulness are in $[0, 1]$; the hallucination rate is also in $[0, 1]$ but lower is better.

Table 1: Example evaluation scores.

Dimension	Score
Coverage	0.87
Faithfulness	0.92
Usefulness	0.85
Hallucination rate	0.08

4 Data Persistence

4.1 SQLite Schema

The backend persists all job data in a single SQLite database opened in WAL mode (Figure 7). Three tables are used: `JOBS` (primary record), `STATES` (full serialised pipeline state), and `EVENTS` (append-only SSE event log).

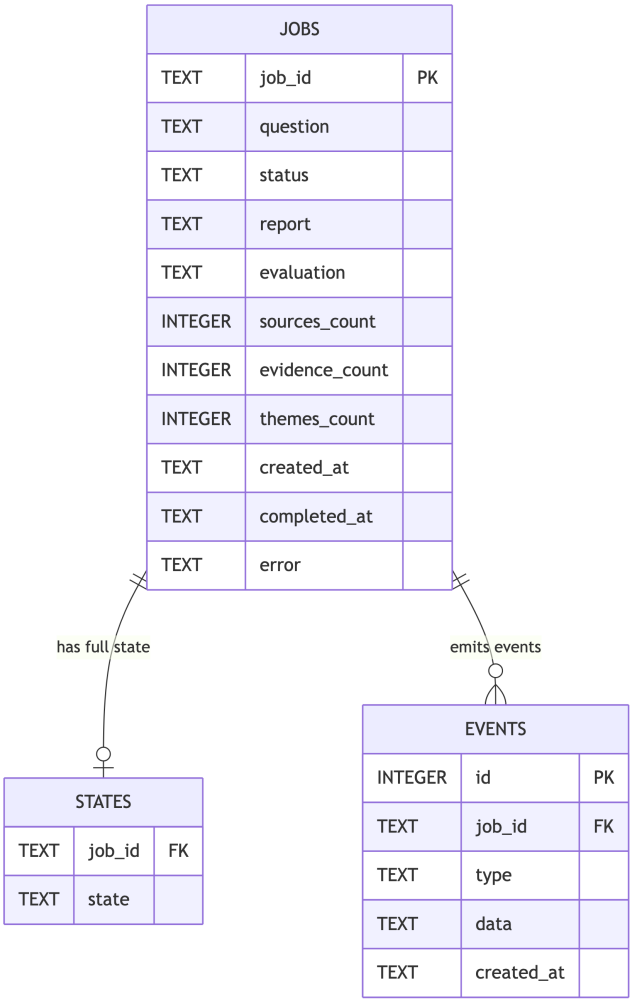


Figure 7: Entity-relationship diagram.

4.2 Job Status Lifecycle

Figure 8 shows the finite-state machine for a research job. A job progresses through *pending* → *searching* → *synthesising* → *reporting* → *evaluating* → *completed*, or transitions to *failed* on any unhandled exception.

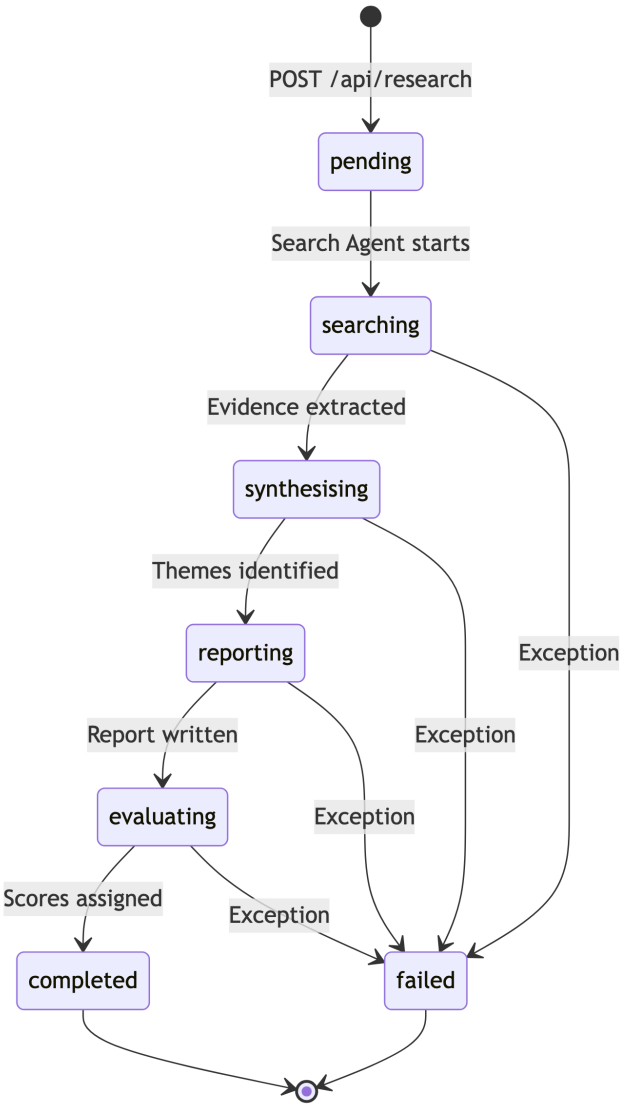


Figure 8: Job status state machine.

4.3 SSE Event Stream

Figure 9 shows the Server-Sent Events (SSE) protocol between the frontend and the `/api/research/{id}/events` endpoint.

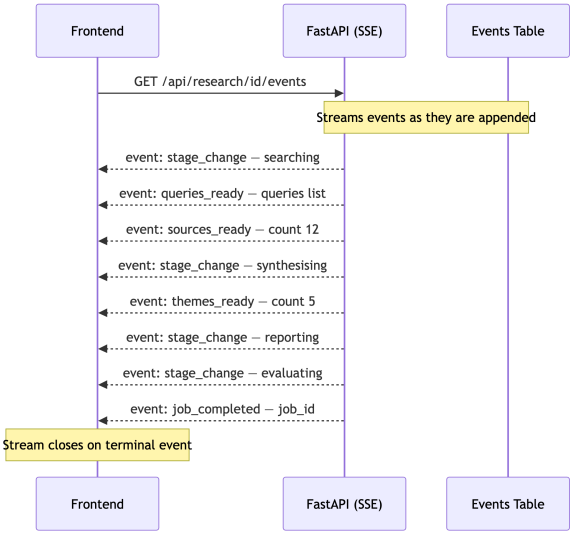


Figure 9: SSE event stream sequence.

5 REST API Reference

All endpoints except `/api/health` require the header `X-API-Key: <key>` when `API_KEY` is configured.

Table 2: REST API endpoints.

Method	Endpoint	Status
POST	/api/research	202
GET	/api/research	200
GET	/api/research/{id}	200
GET	.../{id}/reasoning	200
GET	.../{id}/events	200
GET	.../{id}/pdf	200
DELETE	/api/research/{id}	200
DELETE	/api/research	200
GET	/api/health	200

6 Frontend Application

The Next.js 16 frontend (React 19, Tailwind CSS v4, TypeScript) provides three pages: Home (/), Job History (`/history`), and Job Detail (`/job/[id]`). Figure 10 shows the page and component tree; Figure 11 shows the frontend data flow.

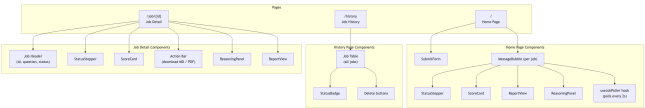


Figure 10: Frontend page and component tree.

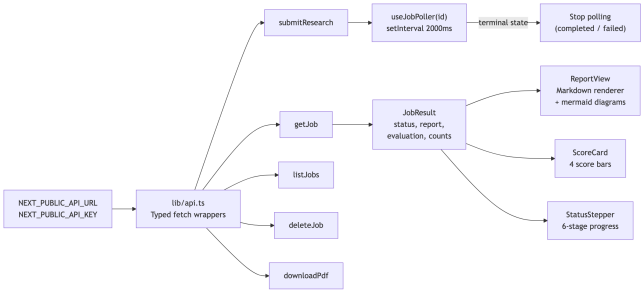


Figure 11: Frontend data flow.

7 Technology Stack

Table 3: Technology stack summary.

Layer	Technology	Role
Backend lang.	Python 3.11+	Agent & API logic
API framework	FastAPI ≥0.115	HTTP, routing
ASGI server	Uvicorn ≥0.30	Production server
LLM (primary)	Groq SDK	llama-3.3-70b
LLM (fallback)	HuggingFace Hub	Llama-3.3-70B
Web search	ddgs ≥9.0	DuckDuckGo
Web search (opt)	tavily-python	Premium search
Scraping	httpx + bs4	Fetch & parse
Validation	Pydantic ≥2	Schemas
Database	SQLite (WAL)	Persistence
PDF export	WeasyPrint ≥62	MD → PDF
Frontend	Next.js 16.2.1	React 19 App Router
Styling	Tailwind CSS v4	Utility-first CSS
Language (FE)	TypeScript	Type safety
Deployment (FE)	Vercel	Hosted frontend

8 Configuration

Key environment variables (set in `api/.env`) are listed in Table 4.

Table 4: Key environment variables.

Variable	Default	Purpose
`LLM_PROVIDER`	auto	groq / huggingface / auto
`GROQ_API_KEY`	—	Primary Groq key
`GROQ_API_KEY_2`	—	Fallback Groq key
`GROQ_MODEL`	llama-3.3-70b	Groq model
`HF_TOKEN`	—	HuggingFace token
`TEMPERATURE`	0.3	LLM temperature
`TAVILY_API_KEY`	—	Premium search
`MAX_SEARCH_QUERIES`	3	Queries per question
`MAX_RESULTS_PER_QUERY`	5	Results per query
`MAX_CONTENT_LENGTH`	3000	Chars per page
`API_KEY`	—	Auth key
`DB_PATH`	data/jobs.db	SQLite location
`CORS_ORIGINS`	Vercel URL	Allowed origins

9 Deployment Architecture

9.1 Current vs. Scalable Architecture

Figure 12 contrasts the current single-instance deployment (FastAPI + SQLite + background threads on a single process) with a production-scale horizontal architecture (load balancer, multiple FastAPI workers, Redis job queue, Celery workers, and PostgreSQL).

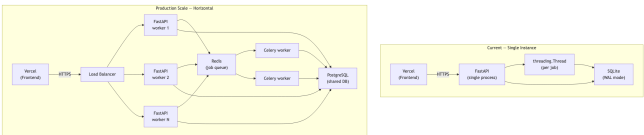
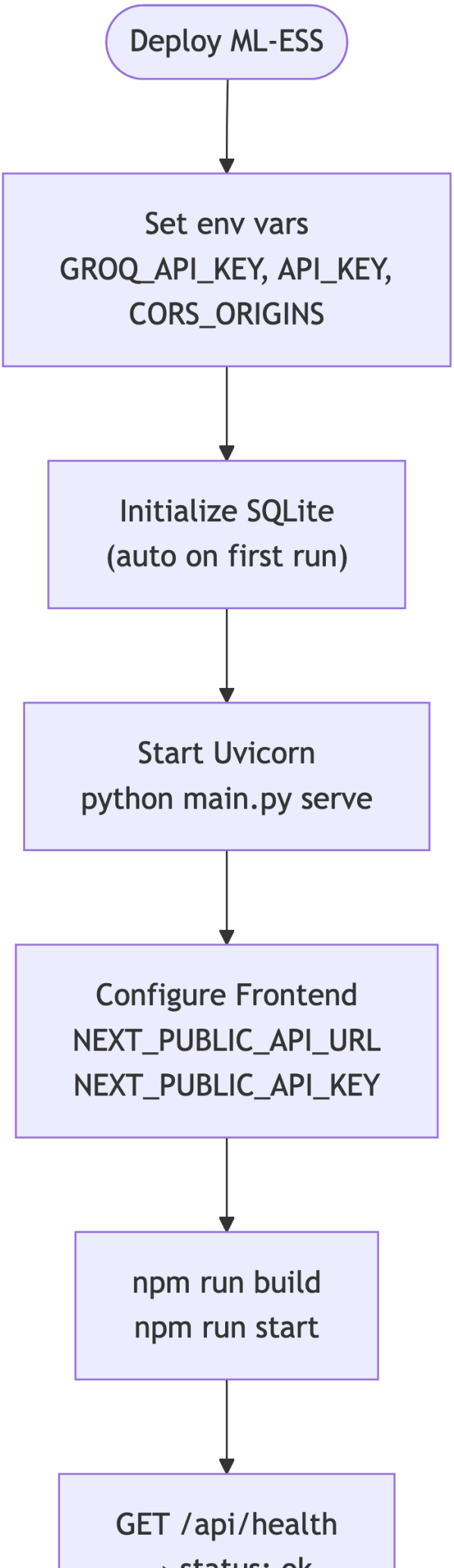


Figure 12: Current vs. scalable deployment architecture.

9.2 Deployment Checklist

Figure 13 summarises the five-step deployment procedure: set environment variables, initialise SQLite, start Uvicorn, configure the frontend, and verify the health endpoint.



10 Conclusion

ML-ESS demonstrates how a small, well-structured multi-agent system can automate a complex, knowledge-intensive task end-to-end. By composing four focused agents — each with a clear role — the pipeline transforms a single natural language question into a cited, evaluated research report in minutes.

The system is designed for clarity and correctness: low LLM temperature for deterministic outputs, retry/fallback logic for resilience, persistent job state for observability, and a clean typed API for frontend integration. The modular agent design makes the system straightforward to extend: adding a new research phase (e.g., a fact-checking agent or a translation layer) requires only inserting a new agent function into the pipeline and extending `SharedState` with the new fields.

Acknowledgements

The author thanks the AIMS Research Innovation Centre and the organisers of the 2026 Doctoral Training School for providing the research environment and computational resources that made this work possible.

References

[1] S. Ramirez, *FastAPI*, 2019. https://fastapi.tiangolo.com

[2] Vercel, *Next.js Documentation*, 2024. https://nextjs.org/docs

[3] Groq Inc., *Groq API — llama-3.3-70b-versatile*, 2024. https://console.groq.com

[4] AI Meta, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[6] CourtBouillon, *WeasyPrint*, 2024. https://weasyprint.org